Task 6 – Organise the Logic into Functions

Python Programming • Assignment 1 • Village Milk Co-operative Payment Calculator | Marks: 4 | Bloom's level: K3 – Apply

Name: **Dhaarani D** Register No.: _____ GitHub: _____ (folder Task06)

1. Functions, parameters and return values

Function	Parameters	Returns	Purpose
rate_per_litre(fat, snf)	fat %, SNF % (float)	rate in Rs (float, 2 dp)	applies the step rules to BASE_RATE
payout(litres, fat, snf)	litres, fat, SNF	amount in Rs (float, 2 dp)	litres × rate, rounded to paise
ten_day_total(entries)	list of (litres, fat, snf) tuples	total in Rs	adds the payout of every entry

functions.py

```
BASE_RATE = 30.00
FAT_STEP_VALUE = 0.40
SNF_STEP_VALUE = 0.30
STD_FAT, STD_SNF, STEP = 3.5, 8.5, 0.1


def rate_per_litre(fat, snf):
    """Return the price per litre (float, 2 decimals) for the given fat and SNF %."""
    fat_steps = round((fat - STD_FAT) / STEP)
    snf_steps = round((snf - STD_SNF) / STEP)
    return round(BASE_RATE + fat_steps * FAT_STEP_VALUE + snf_steps * SNF_STEP_VALUE, 2)


def payout(litres, fat, snf):
    """Return the amount payable (float, 2 decimals) for one entry."""
    return round(litres * rate_per_litre(fat, snf), 2)


def ten_day_total(entries):
    """Return the total payable for a list of (litres, fat, snf) tuples."""
    total = 0.0
    for litres, fat, snf in entries:
        total += payout(litres, fat, snf)
    return round(total, 2)


def show_payout(litres, fat, snf):
    """PRINT a payout (returns None) - cannot be reused in further calculation."""
    print(f"pay {payout(litres, fat, snf):.2f}")


def bad_change_constant():
    BASE_RATE = 99.0            # creates a NEW local name; the global is untouched
    return BASE_RATE


if __name__ == "__main__":
    entries = [(10, 3.5, 8.5), (8.5, 4.2, 8.7), (12, 3.1, 8.3)]
    print("ten_day_total :", ten_day_total(entries))
    r = show_payout(8.5, 4.2, 8.7)
    print("show_payout returned:", r)
    print("payout returned     :", payout(8.5, 4.2, 8.7), "(a value we can add up)")
    print("local BASE_RATE     :", bad_change_constant(), " global BASE_RATE:", BASE_RATE)
    print("rate_per_litre doc  :", rate_per_litre.__doc__)
```

Output

```
ten_day_total : 917.5
pay 283.90
show_payout returned: None
payout returned     : 283.9 (a value we can add up)
local BASE_RATE     : 99.0  global BASE_RATE: 30.0
```

```
rate_per_litre doc  : Return the price per litre (float, 2 decimals) for the given fat and SNF %.
```

2. Returning a value versus printing it

`payout()` **returns** 283.9, a value the caller can store, add up, test with assert or print in any format. `show_payout()` only **prints**; it returns None, so its result cannot be added to a total or checked in a test. Calculation functions should return; only the outermost part of the program should print.

3. Local and global names

Names created inside a function (fat_steps, snf_steps) are **local**: they exist only during the call, so different functions cannot disturb each other. The rate constants are **global** and a function may *read* them freely. If a function assigns to the same name (`BASE_RATE = 99.0` in bad_change_constant) Python creates a *new local* variable and the global stays 30.0, exactly as the output shows (local 99.0, global 30.0). Changing a global from inside a function needs the `global` keyword, which should be avoided because it makes the rate depend on which function ran last.

4. Docstrings

Every function starts with a one-line docstring stating what it returns; `help(payout)` or `payout.__doc__` shows it (last line of the output above).

Alternative Concept and Complexity Analysis

Alternative 1 – one long script: the rate formula pasted inside the loop. **Alternative 2 – lambda expressions:** `rate_l = lambda fat, snf: ...`.

Measured (overhead.py, 150,000 entries, best of 5)

```
functions        177.8 ms for 150000 entries   total 68868741.06
inline script    161.4 ms for 150000 entries   total 68868741.06
lambda           179.4 ms for 150000 entries   total 68868741.06
```

	Functions (chosen)	Inline script	Lambda
Repeated code	none – formula written once	formula copied into every place that needs it; a rate change must be repeated in each copy	none, but one-line only; no docstring, no statements
Time, 150,000 entries	≈ 178 ms	≈ 161 ms	≈ 179 ms
Time per entry	O(1)	O(1)	O(1)
Space	O(1) extra (one stack frame per call)	O(1)	O(1)
Testable on its own	yes	no	awkward (anonymous)

Calling two functions per entry adds about 110 ns per entry (≈ 10 %), i.e. 16 ms for the whole district union (150,000 entries); this is not noticeable. Lambda and def are equally fast because a lambda *is* a function. Functions are therefore chosen: no measurable cost, but no repeated code and each part can be tested separately.

Evidence

Well-named functions with parameters, return values and docstrings, plus an explanation of scope shown by running code. Source: Task06/functions.py, overhead.py.